

Edge vs Cloud vs Onboard Inference for Human Detection in Search and Rescue Drones

A Mini Project Report

Submitted by

Gopichand Sabbavarapu [202302023118]

Kuldeep Das [202302021062]

Arun Daimary [202302021080]

Under the supervision of

Dr. Ranjan Patowary

Assistant Professor

B. Tech. 6th Semester



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CENTRAL INSTITUTE OF TECHNOLOGY KOKRAJHAR

(Deemed to be University under MoE, Govt. of India)

May 2026

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CENTRAL INSTITUTE OF TECHNOLOGY, KOKRAJHAR**

(Deemed to be University under MoE, Govt. of India)

Kokrajhar – 783370, BTR, Assam, India

CERTIFICATE OF APPROVAL

This is to certify that the Mini Project entitled “**Edge vs Cloud vs Onboard Inference for Human Detection in Search and Rescue Drones**” has been carried out by Gopichand Sab-bavarapu (202302023118), Kuldeep Das (202302021062), and Arun Daimary (202302021080) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering from Central Institute of Technology Kokrajhar during the academic year 2025–2026.

(Signature)

(Signature)

Project Guide

Dr. Ranjan Patowary
Assistant Professor, Dept. of CSE
CIT Kokrajhar

Head of Department

Prof. (Name)
Head, Dept. of CSE
CIT Kokrajhar

Date: _____

Place: Kokrajhar, Assam

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CENTRAL INSTITUTE OF TECHNOLOGY, KOKRAJHAR**

(Deemed to be University under MoE, Govt. of India)

Kokrajhar – 783370, BTR, Assam, India

Certificate by the Board of Examiners

This is to certify that the project work entitled “**Edge vs Cloud vs Onboard Inference for Human Detection in Search and Rescue Drones**” submitted by Gopichand Sabbavarapu (202302023118), Kuldeep Das (202302021062) and Arun Daimary (202302021080) to the Department of Computer Science & Engineering of Central Institute of Technology Kokrajhar has been examined and evaluated. This project work has been prepared as per the regulations of Central Institute of Technology and has been qualified for acceptance.

(Signature)

(Signature)

Project Coordinator

Head of the Department

Dr. Ranjan Patowary

Dr. (Head, Dept. of CSE)

Assistant Professor, Dept. of CSE

CIT Kokrajhar

CIT Kokrajhar

ABSTRACT

Search and Rescue (SAR) operations are among the most time-critical activities where the rapid identification of victims can determine life or death outcomes. Recent advancements in drone technology and computer vision have enabled the development of automated aerial monitoring systems capable of detecting humans in large and challenging terrains. However, deploying deep learning models for real-time human detection on drone platforms introduces significant challenges related to computational limitations, network latency, and energy consumption.

This project presents a comprehensive comparative analysis of three distinct inference architectures for human detection in SAR drone systems: onboard inference, edge inference, and cloud inference. A YOLO-based human detection model — specifically YOLOv8 Nano (YOLOv8n) — is deployed consistently across all three system configurations: a Raspberry Pi 3 Model B (onboard), an edge computing laptop (Intel i3), and cloud infrastructure. The system is evaluated based on four key metrics: inference latency, detection accuracy (Precision and Recall), network transmission delay, and power consumption of the Raspberry Pi under different deployment scenarios.

The YOLOv8n model was trained on a curated subset of the VisDrone dataset, retaining only the human-relevant annotations. Both VisDrone class 0 (pedestrian) and class 1 (people) were retained and **merged into a single class** (class 0, labelled *human*) to simplify the detection task. The official VisDrone test split was used for evaluation to ensure reproducibility. The trained model was exported to ONNX format for deployment across all three scenarios. Experimental results indicate that edge inference provides an optimal balance between latency, accuracy, and energy efficiency, making it the most suitable approach for real-world SAR drone deployments.

Keywords: Search and Rescue, UAV, YOLOv8n, ONNX Runtime, Edge Inference, Cloud Inference, Onboard Inference, VisDrone, Human Detection, Latency, Power Consumption.

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to our project guide, Dr. Ranjan Patowary, Assistant Professor, Department of Computer Science and Engineering, Central Institute of Technology Kokrajhar, for his invaluable guidance, constant encouragement, and constructive suggestions throughout the course of this project. His expertise and insights greatly enhanced the quality of our work.

We are also deeply thankful to the Head of the Department and all faculty members of the Department of Computer Science and Engineering for providing the necessary infrastructure and academic environment required for the successful completion of this project. We extend our heartfelt appreciation to the Central Institute of Technology Kokrajhar for providing the laboratory facilities, computing resources, and the supportive ecosystem that made this research possible.

We are grateful to the creators of the VisDrone dataset for making their aerial imagery dataset publicly available, which served as the foundation for training and evaluating our human detection models. We also acknowledge the open-source communities behind YOLO, ONNX Runtime, OpenCV, and Python for their invaluable tools and frameworks. Finally, we thank our family and friends for their unwavering support, encouragement, and motivation throughout this journey.

Gopichand Sabbavarapu [202302023118]

Kuldeep Das [202302021062]

Arun Daimary [202302021080]

Contents

Abstract	3
Acknowledgements	4
1 Introduction	9
1.1 Background	9
1.2 Problem Statement	10
1.3 Objectives	10
1.4 Contributions	10
1.5 Report Organisation	11
2 Literature Review	12
2.1 Drone-Based Object Detection	12
2.2 Onboard Inference on Edge Devices	13
2.3 Edge and Cloud Offloading	15
2.4 Comparative Inference Studies	15
2.5 Research Gap	16
3 System Design and Architecture	17
3.1 System Overview	17
3.2 Dataset Preparation	18
3.2.1 VisDrone Dataset Overview	18
3.2.2 Class Filtering	18
3.2.3 Dataset Cleaning	18
3.2.4 Train / Validation / Test Split	19
3.3 Model Training	19
3.3.1 Training Environment	19
3.3.2 Training Configuration	20
3.3.3 Training Results	20
3.3.4 Confusion Matrix Analysis	21
3.3.5 ONNX Export	22
3.4 Common Preprocessing Pipeline	22
3.5 Onboard Inference Scenario	22
3.6 Edge Inference Scenario	22

3.7	Cloud Inference Scenario	22
3.8	YOLOv8n Model Specification	23
4	Implementation	24
4.1	Tools and Libraries	24
4.2	Onboard Implementation	24
4.3	Edge Implementation	24
4.4	Cloud Implementation	25
4.5	Evaluation Pipeline	25
4.5.1	Latency Evaluation	25
4.5.2	Accuracy Evaluation	26
5	Results and Discussion	27
5.1	Experimental Setup	27
5.2	Latency Analysis	28
5.3	Detection Accuracy	30
5.4	Power Consumption	31
5.5	Cross-Scenario Comparative Analysis	31
5.6	Suitability for Real-World SAR Operations	32
6	Conclusion and Future Work	34
6.1	Conclusion	34
6.2	Limitations	34
6.3	Future Work	35

List of Tables

3.1	Experimental Hardware and Software Configuration	17
3.2	VisDrone Dataset Split Sizes After Class Filtering	19
3.3	YOLOv8n Training Hyperparameters	20
3.4	YOLOv8n Validation Performance After Training	20
3.5	Validation Set Detection Statistics (Best Epoch 87)	21
3.6	YOLOv8n Model Specification (Identical Across All Scenarios)	23
4.1	Tools and Libraries Used	24
4.2	Latency Metric Definitions	25
5.1	Experimental Hardware and Software Configuration	27
5.2	Mean Latency Breakdown Across Scenarios (milliseconds)	28
5.3	Detection Accuracy Results (VisDrone Test Split, IoU = 0.50)	30
5.4	Raspberry Pi 3 Model B Power Consumption Across Scenarios	31
5.5	Summary Comparison — Onboard vs. Edge vs. Cloud	32

List of Figures

3.1	YOLOv8n training loss and validation mAP curves over epochs	21
3.2	YOLOv8n confusion matrix on the VisDrone validation set (best epoch 87) . .	21
5.1	Mean total latency comparison across the three inference scenarios	28
5.2	Mean latency breakdown by component (preprocessing, network delay, inference) for each scenario	29
5.3	Per-image total latency across the VisDrone test images for all three scenarios. Cloud latency exhibits significant spikes due to internet-path variability over the Tailscale VPN.	30

Chapter 1

Introduction

This project investigates and experimentally compares three inference paradigms — onboard, edge, and cloud — in the context of human detection for Search and Rescue (SAR) drone systems using the YOLOv8 Nano (YOLOv8n) model deployed consistently across all three configurations.

1.1 Background

Search and Rescue (SAR) operations are among the most time-sensitive emergency response activities, where the rapid identification and location of survivors can be the deciding factor between life and death. Traditional SAR methods heavily rely on manual observation by ground teams and manned aerial vehicles, which are often slow, expensive, and limited in their ability to cover large or inaccessible terrains such as dense forests, flood-affected zones, earthquake rubble, and mountainous regions.

The advent of Unmanned Aerial Vehicles (UAVs), commonly known as drones, has fundamentally transformed SAR operations. Drones equipped with high-resolution cameras and navigation systems can rapidly survey large areas, capture aerial imagery in real time, and reach locations that are inaccessible or dangerous for human rescuers. Their ability to operate autonomously or semi-autonomously makes them ideal tools for large-scale aerial monitoring.

However, the full potential of drone-based SAR systems is realised only when they are coupled with intelligent computer vision algorithms capable of automatically detecting humans in aerial images. Deep learning-based object detection models, particularly the YOLO (You Only Look Once) family of architectures, have demonstrated remarkable performance in real-time object detection tasks. Despite these advances, deploying deep learning models in operational drone platforms presents significant engineering challenges. Embedded computing devices such as the Raspberry Pi have limited computational resources, while offloading computation to edge or cloud resources introduces communication latency and network dependency.

1.2 Problem Statement

Modern deep learning models achieve high detection accuracy for human detection in aerial imagery. However, their computational demands are often incompatible with the resource-constrained embedded systems deployed on drone platforms. Running full-precision YOLO inference directly on a Raspberry Pi significantly increases latency and power consumption, while offloading computation to edge or cloud resources introduces communication overhead and network dependency.

The core challenge addressed in this project is: *How do onboard, edge, and cloud inference architectures compare in terms of inference latency, detection accuracy, network transmission delay, and power consumption for real-time human detection in SAR drone systems?* Answering this question requires careful design, implementation, and experimental evaluation of all three architectures within a unified framework using consistent models, datasets, and evaluation protocols.

1.3 Objectives

The specific objectives of this project are as follows:

- Curate and prepare a VisDrone-based dataset filtered to human detection classes (pedestrian and people merged into a single human class) for training and evaluation.
- Train a YOLOv8n model on the prepared dataset and export it to ONNX format.
- Design and implement three distinct inference architectures: onboard (Raspberry Pi 3 Model B), edge (local laptop via Wi-Fi), and cloud (remote server via Internet/VPN).
- Export the trained YOLOv8n model to ONNX format for cross-platform deployment across all three scenarios.
- Evaluate each architecture based on inference latency, detection accuracy (Precision and Recall), network transmission delay, and power consumption.
- Analyse the trade-offs between computational efficiency, detection quality, network dependency, and energy usage.
- Identify the most suitable deployment strategy for real-world SAR operations.

1.4 Contributions

- A novel curated subset of the VisDrone dataset with pedestrian and people classes merged into a single human class, with documented cleaning and preparation methodology.
- A trained YOLOv8n model for aerial human detection, validated on the official VisDrone test split.
- A unified comparative framework for evaluating onboard, edge, and cloud inference

architectures.

- Implementation of a complete YOLO-based detection pipeline using Python, OpenCV, and ONNX Runtime.
- Quantitative analysis of latency, accuracy, network delay, and power consumption to guide real-world deployment decisions.

1.5 Report Organisation

The remainder of this report is organised as follows. Chapter 2 presents a review of related literature covering drone-based detection, edge/cloud inference, and prior comparative studies. Chapter 3 describes the dataset preparation, model training, and system design for all three scenarios. Chapter 4 details the implementation, tools, and protocols used. Chapter 5 presents the experimental results and discussion. Chapter 6 concludes the report and outlines directions for future work.

Chapter 2

Literature Review

The integration of deep learning-based object detection with unmanned aerial vehicle (UAV) platforms has opened new frontiers in Search and Rescue (SAR) operations. Deploying such models on drone systems, however, raises fundamental engineering questions about *where* inference should occur: on the drone itself, at a nearby edge server, or in a remote cloud. This chapter reviews the key literature underpinning this project across four thematic areas: drone-based object detection and the YOLO family of architectures; onboard inference on resource-constrained embedded devices; edge and cloud offloading architectures; and comparative inference studies. The chapter concludes by identifying the specific research gaps that this project addresses.

2.1 Drone-Based Object Detection

The use of drones equipped with cameras for automated person detection has attracted significant research interest over the past decade. Drones offer compactness, mobility, relatively low cost, and high-resolution real-time video recording, making them standard tools in SAR services globally [2]. However, automated detection of humans in drone imagery presents unique challenges absent from ground-level benchmarks: objects appear very small relative to the scene (often only a few tens of pixels), occur at highly variable scales due to altitude changes, suffer from occlusion by vegetation or terrain, and appear in atypical postures when exhausted or injured [10].

The VisDrone benchmark, introduced by Zhu et al. (2022), has become the standard evaluation dataset for drone-based object detection [2]. Comprising over 10,000 images captured by drones across 14 Chinese cities at altitudes from 5 m to 100 m, it contains 10 object categories including pedestrians and people. The dataset exposes key challenges: the majority of annotations are classified as “small” (bounding box area $< 32^2$ px), making detection considerably harder than on ground-level benchmarks such as MS COCO [2]. The present project uses a filtered subset of VisDrone retaining only the two human-relevant classes (pedestrian and people, merged into a single *human* class), directly motivated by the SAR context.

Among object detection architectures, the YOLO (You Only Look Once) family has emerged as the dominant paradigm for real-time detection tasks due to its single-stage design that

simultaneously predicts bounding boxes and class probabilities across a grid, enabling significantly faster inference than two-stage detectors such as Faster R-CNN [6]. Sambolek and Ivasic-Kos (2021) conducted a rigorous comparative evaluation of Faster R-CNN, YOLOv4, RetinaNet, and Cascade R-CNN on both the VisDrone benchmark and their custom SARD dataset — a collection of 1,981 annotated drone images depicting actors in positions typical of injured or exhausted persons in non-urban terrain [10]. After training and evaluation, YOLOv4 achieved the best performance across all metrics: 61% average precision (AP), 96% AP at IoU = 0.5, and the lowest false positive rate. The authors introduced the ROpti metric, defined as $(TP - FP)/(TP + FN)$, to balance detection completeness against false positive cost, reaching an ROpti value of 0.928 at a network resolution of 832×832 [10]. This work strongly supports the selection of YOLO-family models for SAR applications and motivates the use of YOLOv8 in the present project.

YOLOv8, released by Ultralytics in 2023, advances the YOLO lineage through architectural improvements including decoupled classification and regression heads, an anchor-free detection strategy, and the C2f (Cross Stage Partial with two convolutions) backbone module that improves gradient flow during training [1]. The YOLOv8 Nano (YOLOv8n) variant, with approximately 3.2 M parameters and a model size of around 6 MB in ONNX FP32 format, achieves competitive accuracy while remaining deployable on resource-constrained hardware. Basireddy et al. (2025) demonstrated that YOLOv9 — a further architecture in the YOLO lineage — combined with EfficientDet in an ensemble achieves high precision on the Heridal and Aerial Rescue Object Detection datasets for SAR human detection, reinforcing that the YOLO family remains state-of-the-art for this application domain [16].

Sambolek and Ivasic-Kos (2021) also examined the critical relationship between drone altitude and detection reliability, finding that accurate detection required persons to occupy at least 100×100 pixels in the image — corresponding to drone altitudes no greater than approximately 30 m with their DJI Phantom 4A camera [10]. Above 45–50 m, detection rates dropped substantially. Weather robustness experiments using an augmented dataset (fog, snow, frost, motion blur) showed that AP50 fell from 96% on clean images to 55–68% under adverse conditions, a finding directly relevant to real-world SAR deployment where environmental conditions cannot be controlled [10].

2.2 Onboard Inference on Edge Devices

Running deep learning models directly on resource-constrained embedded hardware remains a significant challenge. Embedded platforms such as the Raspberry Pi 3 Model B (ARM Cortex-A53 @ 1.2 GHz, no GPU, 1 GB LPDDR2 RAM) offer low power consumption and sufficient computational capacity for lightweight tasks, but their CPU-only inference throughput is substantially lower than GPU-accelerated systems.

Raviprasad et al. (2024) deployed a lightweight SSD MobileNet V3 model on a Raspberry Pi attached to an F450 quadcopter for real-time human detection in SAR operations [14]. The system processed live video feeds frame-by-frame via the COCO-pretrained model, achieving 67% detection accuracy. The authors explicitly attributed this limitation to the resource-constrained nature of the Raspberry Pi: without a GPU or neural processing unit (NPU), the model cannot exploit hardware parallelism and throughput is severely limited by the CPU clock speed. Their work validates that onboard inference on single-board computers is *feasible* for SAR, while simultaneously demonstrating the accuracy-efficiency trade-off inherent in this deployment mode [14]. This result establishes a practical baseline against which the YOLOv8n onboard inference evaluated in the present project can be contextualised.

A common approach to reducing the computational burden of deep learning models for edge deployment is model compression. Han et al. (2016) demonstrated that deep networks can be compressed by up to $49\times$ through a combination of weight pruning, trained quantization, and Huffman coding with minimal accuracy loss [7]. Quantization — reducing parameter precision from 32-bit floating point (FP32) to 8-bit integer (INT8) — is particularly relevant for CPU-bound inference: INT8 models can run $2\text{--}4\times$ faster than FP32 equivalents on hardware that supports vectorised integer arithmetic [7]. The present project deploys the YOLOv8n model in FP32 ONNX format across all three scenarios, establishing FP32 baselines that future work could improve through INT8 quantization.

The ONNX (Open Neural Network Exchange) format, supported by the ONNX Runtime inference engine, provides a hardware-agnostic model representation that enables consistent deployment across CPUs (onboard and edge scenarios) and GPUs (cloud scenario) via execution providers [3]. This cross-platform compatibility is central to the present project's unified comparison methodology. Howard et al. (2017) introduced MobileNets, lightweight CNN architectures based on depthwise separable convolutions that reduce computation and model size while maintaining competitive accuracy for mobile and embedded deployment [8]. MobileNet-based architectures have been widely adopted as backbones for onboard drone detection systems, including the SSD MobileNet V3 used by Raviprasad et al. (2024) [14]. YOLOv8n occupies a similar design philosophy — prioritising efficiency — within the YOLO family [1].

Hossain et al. (2023) surveyed the integration of multimodal AI with edge computing for real-time decision-making, identifying model pruning, quantization, knowledge distillation, and federated learning as key techniques for deploying computationally demanding models on resource-constrained edge devices [15]. Their taxonomy of edge AI deployment challenges — resource limitation, heterogeneous hardware, security and privacy, continual learning, and bandwidth constraints — maps directly onto the practical difficulties encountered in the onboard inference pipeline of the present project.

2.3 Edge and Cloud Offloading

Offloading computation from a resource-constrained drone to a more powerful external device — whether a nearby edge server or a remote cloud — is a well-established strategy for enabling low-latency, high-accuracy inference without the power and thermal penalties of onboard processing.

Koubaa et al. (2023) presented AERO, a multi-stage deep learning system that deploys YOLOv7 and YOLOv4 directly on an NVIDIA Jetson Xavier NX mounted onboard a custom hexacopter, with the cloud used exclusively for non-time-critical functions such as data storage and mission orchestration [11]. A key quantitative finding from AERO is that cloud connectivity alone reduces inference throughput from 12.3 FPS to 5.0 FPS — a 59% degradation attributable entirely to network communication overhead [11].

Koubaa and Qureshi (2018) investigated cloud-based real-time UAV tracking in DroneTrack, a system that routes GPS coordinates from a moving target through a cloud intermediary to a drone operating in GUIDED mode over MAVLink and WebSocket protocols [12]. Experimental evaluation demonstrated that the system achieved an average tracking accuracy of 3.5 m for slow-moving targets, but that a single 10-second communication gap caused the tracking distance to reach 34 m — illustrating the fragility of purely cloud-dependent real-time UAV control loops [12].

McEnroe, Wang, and Liyanage (2022) conducted a comprehensive survey of edge AI for UAVs, noting that cloud AI introduces significantly long transmission latency, while edge AI enables the majority of processing at a nearby edge server with a substantially shorter transmission delay [13]. Their seven-level edge AI categorisation framework provides a useful conceptual mapping: the Raspberry Pi onboard scenario corresponds to fully on-device inference, while the edge (local laptop via Wi-Fi WebSocket) and cloud (Tailscale VPN over internet) scenarios correspond to cloud-edge co-inference levels [13].

2.4 Comparative Inference Studies

Several works have compared inference paradigms across different deployment configurations, though none provide the specific three-way comparison that the present project addresses.

Koubaa et al. (2023) provide the closest existing comparison in AERO, quantifying the throughput difference between fully local edge inference and cloud-coupled inference for the same model [11]. However, this comparison is made within a single proprietary system architecture and does not evaluate the onboard (CPU-only) paradigm as a third configuration.

Hossain et al. (2023) provide a conceptual comparison of edge and cloud AI deployment but do not provide experimental latency or accuracy measurements for a specific model or dataset [15]. The present project fills this gap by providing empirical measurements across all three paradigms

using a consistent model and protocol stack.

Basireddy et al. (2025) present a sophisticated ensemble SAR detection system but do not report frame-per-second throughput, inference latency, or computational footprint [16]. The absence of latency evaluation is characteristic of a broader pattern in the SAR detection literature: most works optimise for accuracy metrics while neglecting the deployment constraints that determine operational viability on a drone platform.

2.5 Research Gap

Based on the literature reviewed above, the following specific gaps are identified and directly addressed by this project:

- **No three-way unified comparison.** No existing study provides a controlled, three-way comparison of onboard, edge, and cloud inference for SAR drone human detection using a consistent model, codebase, and dataset.
- **VisDrone not used in inference paradigm studies.** Prior comparative inference studies use general-purpose or ground-level benchmarks. The VisDrone aerial dataset has not been used in any three-way inference paradigm comparison.
- **ONNX cross-platform deployment not benchmarked for SAR.** The performance impact of deploying the same ONNX-exported model across onboard CPU (Raspberry Pi), edge CPU (laptop), and cloud GPU (ONNX Runtime CUDA provider) backends has not been systematically compared for SAR human detection.
- **Latency neglected as a primary evaluation metric.** Most SAR detection literature reports accuracy metrics while providing little or no measurement of inference latency or end-to-end pipeline delay.
- **Power consumption comparison absent.** No existing work on inference paradigm comparison for SAR drones measures and compares the power consumption of the drone-side device across onboard, edge, and cloud scenarios.

Chapter 3

System Design and Architecture

This chapter describes the dataset preparation process, model training procedure, the common preprocessing pipeline, and the architecture of each of the three inference scenarios: Onboard, Edge, and Cloud.

3.1 System Overview

The system consists of a Raspberry Pi 3 Model B acting as the client (simulating the drone platform). For the Edge scenario, an Intel Core i3 laptop acts as the inference server over a local Wi-Fi network. For the Cloud scenario, a GPU-enabled laptop equipped with an NVIDIA RTX 5050 GPU and 16 GB RAM acts as a simulated cloud server connected over the public internet via a Tailscale WireGuard VPN. For the Onboard scenario, inference is performed entirely on the Raspberry Pi without any network communication. All three configurations use the same YOLOv8n ONNX model, the same preprocessing pipeline, and are evaluated on the same VisDrone test-dev images.

Table 3.1: Experimental Hardware and Software Configuration

Component	Specification
Client Device	Raspberry Pi 3 Model B, 1 GB RAM, ARM Cortex-A53 @ 1.2 GHz
Edge Server	Intel Core i3 Laptop, 8 GB RAM
Cloud Server	Laptop with NVIDIA RTX 5050 GPU, 16 GB RAM (simulated cloud)
Network (Edge)	Local Wi-Fi, Custom TCP Protocol
Network (Cloud)	Tailscale WireGuard VPN over Internet, Custom TCP Protocol
Dataset	VisDrone — aerial drone imagery, diverse conditions
Detection Classes	Human (merged: pedestrian + people → class 0)
Confidence Threshold	0.45
Image Input Size	640 × 640 pixels

3.2 Dataset Preparation

3.2.1 VisDrone Dataset Overview

The VisDrone dataset was introduced by Zhu et al. (2022) and is maintained by the AISKYEYE Lab at Tianjin University [2]. It comprises 10,209 static images captured by drone-mounted cameras across 14 cities in China, covering a wide range of scenarios including urban streets, parks, markets, and residential areas. Images were captured at altitudes ranging from approximately 5 m to 100 m under varied lighting and weather conditions. Annotations are provided in a bounding-box format covering 10 object categories: pedestrian (class 0), people (class 1), bicycle (class 2), car (class 3), van (class 4), truck (class 5), tricycle (class 6), awning-tricycle (class 7), bus (class 8), and motor (class 9).

For this project, the pre-converted YOLO-format labels (bounding boxes normalised to $[0, 1]$ in `class_id cx cy w h` format) were used directly. The official dataset is divided into three splits: a training set of 6,471 images, a validation set of 548 images, and a test-dev set of 1,610 images. The test-dev split does not include ground-truth labels in the public release; however, since our filtered label files were available for all splits, the test-dev split was used as the evaluation set and is referred to as the test split hereafter.

3.2.2 Class Filtering

The original VisDrone dataset contains 10 object classes. Since this project focuses exclusively on human detection for SAR applications, only two classes are retained: **class 0 (pedestrian)** and **class 1 (people)**. All annotations belonging to the remaining eight classes were removed from the label files.

The dataset preparation script processed each label `.txt` file as follows:

1. Read each annotation line (format: `class_id cx cy w h`).
2. If `class_id` is 0 (pedestrian) or 1 (people), remap to class 0 and retain the line.
3. All other class IDs are discarded.
4. Write the filtered lines to the output label file.

3.2.3 Dataset Cleaning

After class filtering, each output label file was inspected to verify structural integrity. Label files in which all annotations belonged to non-human classes produced empty output files (zero lines); these were retained as valid YOLO negative samples rather than discarded, since images with no humans are legitimate hard negatives that improve the model's specificity during training. No images were removed from any split on the basis of having an empty label file. Annotations with malformed entries (lines not containing exactly five whitespace-separated fields) were silently

discarded by the processing script, consistent with standard YOLO dataset hygiene practice.

Table 3.2: VisDrone Dataset Split Sizes After Class Filtering

Split	Total Images	Used in Experiments
Train	6,471	6,471
Val	548	548
Test	1,610	290*
Total	8,629	7,309

* A subset of 290 images was randomly sampled from the full 1,610-image test split for inference evaluation across all three scenarios (Onboard, Edge, and Cloud), due to the time and resource constraints of running experiments on the Raspberry Pi 3 Model B.

3.2.4 Train / Validation / Test Split

The official VisDrone dataset splits were used without modification. The test split was kept strictly separate and was not seen during training or hyperparameter tuning.

3.3 Model Training

3.3.1 Training Environment

Training was performed on a Windows system equipped with an NVIDIA GeForce RTX 5050 Laptop GPU (8.0 GB VRAM, CUDA-enabled). Mixed precision training (AMP) was enabled to reduce VRAM usage.

3.3.2 Training Configuration

Table 3.3: YOLOv8n Training Hyperparameters

Parameter	Value
Model architecture	YOLOv8n (Nano)
Pretrained weights	yolov8n.pt (COCO pretrained)
Image size	640×640
Epochs	100
Batch size	12
Optimizer	AdamW
Initial learning rate	0.001
LR schedule	Cosine annealing
Weight decay	0.0005
Early stopping patience	20 epochs
Mixed precision (AMP)	Enabled
Workers	1 (Windows stability)
Mosaic augmentation	0.7
MixUp augmentation	0.1
Copy-paste augmentation	0.2

3.3.3 Training Results

Table 3.4: YOLOv8n Validation Performance After Training

Metric	Value
mAP@0.5 (val)	50.92%
mAP@0.5:0.95 (val)	20.40%
Precision (val)	65.78%
Recall (val)	44.77%
Best epoch	87 / 100
Total training time	~2.95 hours

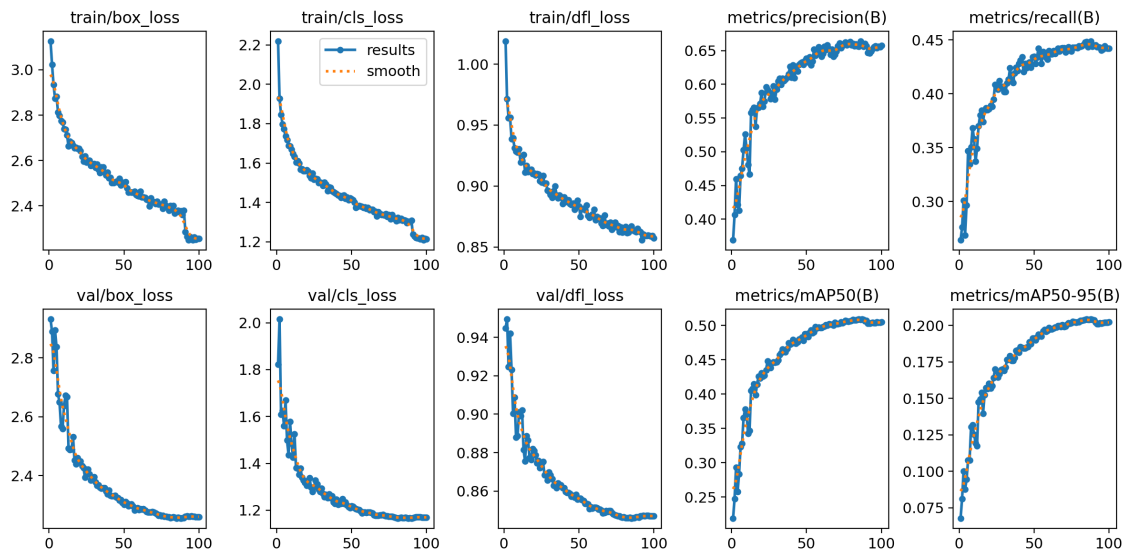


Figure 3.1: YOLOv8n training loss and validation mAP curves over epochs

Table 3.5: Validation Set Detection Statistics (Best Epoch 87)

Metric	Value
True Positives (TP)	6,806
False Negatives (FN)	3,645
False Positives (FP)	7,163
Precision	48.72%
Recall	65.12%
F1-Score	55.74%

3.3.4 Confusion Matrix Analysis

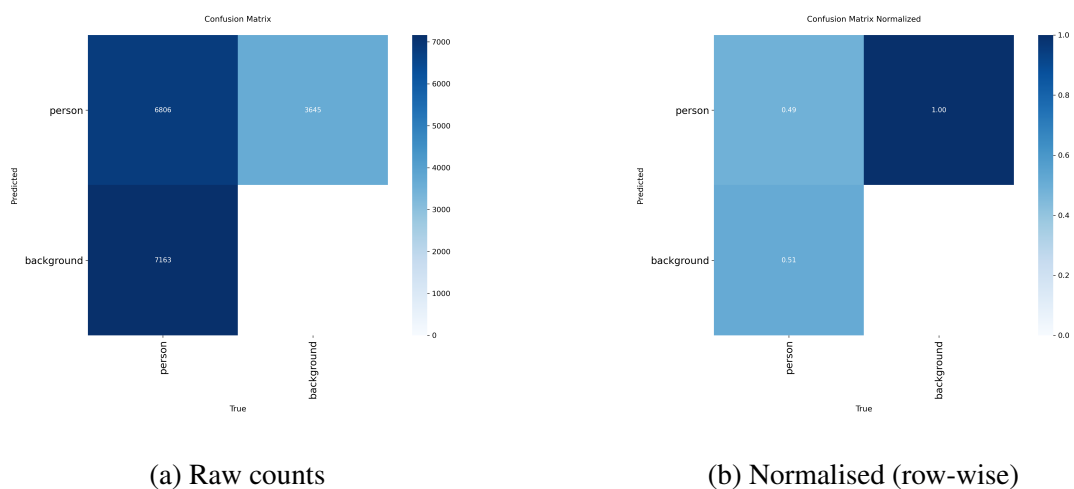


Figure 3.2: YOLOv8n confusion matrix on the VisDrone validation set (best epoch 87)

3.3.5 ONNX Export

After training, the best checkpoint was exported to ONNX format for deployment across all three scenarios via ONNX Runtime:

```
from ultralytics import YOLO
model = YOLO("best.pt")
model.export(format="onnx", imgsz=640, opset=12,
             simplify=True, dynamic=False, nms=True)
```

3.4 Common Preprocessing Pipeline

All three scenarios share an identical preprocessing pipeline executed on the Raspberry Pi before any inference or transmission. Steps: (1) load image, (2) resize to 640×640 , (3) blur detection via Laplacian variance (threshold 100.0; images with variance < 50.0 dropped; 50.0–100.0 range denoised with Gaussian blur), (4) JPEG compress at quality 80 (~ 40 –80 KB per image). Pixel normalisation is applied inside the inference pipeline immediately before the model forward pass.

3.5 Onboard Inference Scenario

In the Onboard scenario, the entire inference pipeline runs locally on the Raspberry Pi 3 Model B. The YOLOv8n ONNX model (FP32) is loaded via ONNX Runtime with `CPUExecutionProvider`. No network communication occurs.

3.6 Edge Inference Scenario

In the Edge scenario, image preprocessing is performed on the Raspberry Pi, which transmits preprocessed images to a nearby edge server (laptop) over local Wi-Fi using a custom binary TCP framing protocol. The edge server runs YOLOv8n inference via ONNX Runtime using `CPUExecutionProvider`. Total latency consists of transmission delay and server-side inference time.

3.7 Cloud Inference Scenario

In the Cloud scenario, the Raspberry Pi transmits preprocessed images to a laptop server over the public internet via Tailscale WireGuard VPN using the same custom binary TCP framing protocol as the Edge scenario. The server runs YOLOv8n via ONNX Runtime with

`CUDAExecutionProvider`. An inference queue decouples network receipt from inference processing.

3.8 YOLOv8n Model Specification

Table 3.6: YOLOv8n Model Specification (Identical Across All Scenarios)

Property	Value
Architecture	YOLOv8n (Nano)
Parameters	~3.2M
ONNX model size	~12 MB (FP32, embedded NMS)
Input size	640×640 px
Output classes	1 (human: merged pedestrian + people)
Confidence threshold	0.45
NMS IoU threshold	0.45

Chapter 4

Implementation

4.1 Tools and Libraries

Table 4.1: Tools and Libraries Used

Tool / Library	Version	Purpose
Python	3.10+	Primary programming language
Ultralytics YOLOv8	Latest	Model training and ONNX export
ONNX Runtime	1.16+	Optimised ONNX inference (CPU and GPU)
OpenCV (cv2)	4.8+	Image loading, resizing, blur detection
NumPy	1.24+	Array operations
socket	Standard Library	Custom TCP communication
Tailscale	Latest	WireGuard VPN for Cloud scenario
pandas	2.0+	CSV merging and latency evaluation

4.2 Onboard Implementation

The Onboard scenario runs entirely on the Raspberry Pi 3 Model B. The YOLOv8n FP32 ONNX model is loaded via ONNX Runtime with `CPUExecutionProvider`. The inference pipeline proceeds as follows: the preprocessed image (640×640 , JPEG-decoded) is normalised to $[0, 1]$ float32, transposed to CHW format, and passed to the ONNX Runtime session. The exported ONNX model includes integrated Non-Maximum Suppression (NMS). Final detections are filtered using a confidence threshold of 0.45 to produce output bounding boxes. Preprocessing and inference times are recorded using `time.perf_counter()` and written to a per-run CSV file for evaluation.

4.3 Edge Implementation

The Edge client runs on the Raspberry Pi and establishes a persistent TCP connection to the laptop server over local Wi-Fi using a custom binary framing protocol. For each image, the

client records a send timestamp (t_{send}), transmits the JPEG-compressed image as binary data, and records an ACK timestamp (t_{ack}) upon receiving the server's detection result. The network round-trip delay is computed as $(t_{\text{ack}} - t_{\text{send}}) \times 1000$ ms. The server receives the image, runs ONNX Runtime inference with `CPUExecutionProvider`, and returns the detection JSON. Client and server CSV files are merged by image ID in the evaluation script to compute per-image total latency.

4.4 Cloud Implementation

The Cloud client establishes a persistent TCP connection to the laptop server over the Tailscale WireGuard VPN. The session follows a structured protocol: `AUTH` $\rightarrow$ `RUN_INIT` $\rightarrow$ `IMAGE` $\times N$ $\rightarrow$ `CSV upload` $\rightarrow$ `GOODBYE`. Each `IMAGE` message uses a custom binary framing protocol: a 4-byte header encodes message type and payload length, followed by the JPEG image bytes. The server maintains an inference queue that decouples network reception from ONNX Runtime processing, allowing the server to continue receiving frames while a previous inference is in progress. The server runs YOLOv8n with `CUDAExecutionProvider`. Exponential-backoff retry logic on the client handles transient VPN connection drops, retrying up to five times with delays of 1, 2, 4, 8, and 16 seconds before aborting the session.

4.5 Evaluation Pipeline

4.5.1 Latency Evaluation

Table 4.2: Latency Metric Definitions

Component	Formula	Measured By
<code>preprocess_ms</code>	$t_{\text{end}} - t_{\text{start}}$ of preprocessing	Client (Pi)
<code>network_delay_ms</code>	$(t_{\text{ACK}} - t_{\text{send}}) \times 1000$	Client (Pi)
<code>inference_latency_ms</code>	$(t_{\text{inf_end}} - t_{\text{inf_start}}) \times 1000$	Server
<code>total_latency_ms</code>	Sum of all components	Computed

All timestamps are recorded using `time.perf_counter()` on their respective hosts. For the Edge and Cloud scenarios, client and server CSV files are merged on `image_id` to produce a unified per-image latency record. The Onboard scenario records all components locally since no network communication occurs.

4.5.2 Accuracy Evaluation

Detection accuracy was evaluated by comparing predicted bounding boxes against YOLO-format ground-truth annotations using Intersection over Union (IoU)-based matching with an IoU threshold of 0.5. For each image, predicted bounding boxes were matched to the unmatched ground-truth box of the same class with the highest IoU if the overlap exceeded the threshold. Each ground-truth box could be matched to at most one prediction.

Matched prediction–ground-truth pairs were counted as True Positives (TP), unmatched predictions were counted as False Positives (FP), and unmatched ground-truth boxes were counted as False Negatives (FN). Per-image precision, recall, and F1-score were computed from these counts. Aggregate metrics were then obtained by summing TP, FP, and FN across all evaluated images before calculating overall precision, recall, and F1-score.

Chapter 5

Results and Discussion

This chapter presents the experimental results for all three inference scenarios — Onboard, Edge, and Cloud — evaluated across three dimensions: inference latency, detection accuracy, and power consumption. All experiments used the same YOLOv8n ONNX model and the same preprocessing pipeline. Evaluation was performed on a subset of 290 images selected from the official VisDrone test split. Results are analysed and compared to identify the most suitable deployment strategy for real-world SAR operations.

5.1 Experimental Setup

All experiments were conducted using a Raspberry Pi 3 Model B (1 GB RAM, ARM Cortex-A53 @ 1.2 GHz) as the client device, simulating the onboard drone platform. An Intel Core i3 laptop (8 GB RAM) served as the inference server for the Edge scenario, while a GPU-enabled laptop with 16 GB RAM and an NVIDIA RTX 5050 GPU served as the inference server for the simulated Cloud scenario. The hardware and software configuration is summarised in Table 5.1.

Table 5.1: Experimental Hardware and Software Configuration

Component	Specification
Client Device	Raspberry Pi 3 Model B, 1 GB RAM, ARM Cortex-A53 @ 1.2 GHz
Edge Server	Intel Core i3 Laptop, 8 GB RAM
Cloud Server	Laptop with NVIDIA RTX 5050 GPU, 16 GB RAM (simulated cloud)
Network (Edge)	Local Wi-Fi, WebSocket Protocol
Network (Cloud)	Tailscale WireGuard VPN over Internet, Custom TCP Protocol
Model	YOLOv8n ONNX FP32, single class (human)
Confidence Threshold	0.45
IoU Threshold	0.50 (NMS and evaluation)
Evaluation Dataset	290 images selected from the official VisDrone test split

5.2 Latency Analysis

Inference latency is the most operationally critical metric for real-time SAR drone systems. Table 5.2 presents the mean latency breakdown across all three scenarios, decomposed into three components: preprocessing time on the Raspberry Pi, network delay, and server-side inference time.

Table 5.2: Mean Latency Breakdown Across Scenarios (milliseconds)

Latency Component	Onboard	Edge	Cloud
Preprocessing time (Pi)	154.41	137.02	136.95
Network delay	0 (local)	33.38	574.82
Inference time	1,809.33	97.56	54.86
Total latency	1,963.75	267.95	766.63

Figure 5.1 shows the mean total latency per scenario. The Edge scenario achieves the lowest mean total latency at 267.95 ms — $7.3\times$ faster than Onboard (1,963.75 ms) and $2.9\times$ faster than Cloud (766.63 ms). Figure 5.2 breaks this total into its constituent components, revealing *why* each scenario performs as it does.

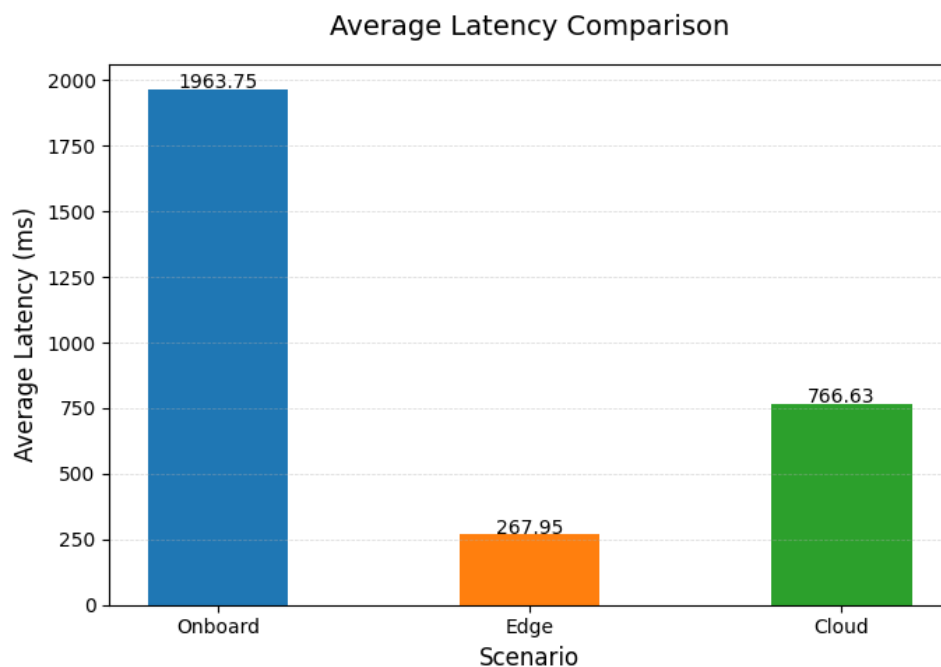


Figure 5.1: Mean total latency comparison across the three inference scenarios

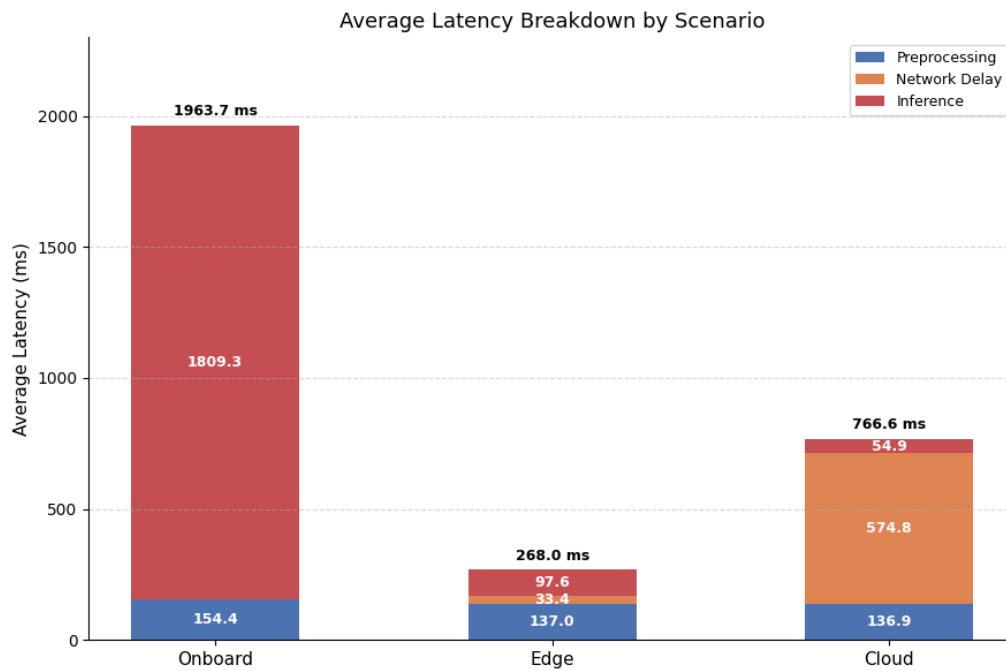


Figure 5.2: Mean latency breakdown by component (preprocessing, network delay, inference) for each scenario

The stacked breakdown in Figure 5.2 makes the bottleneck of each scenario immediately clear. In the Onboard scenario, inference accounts for 1,809.33 ms out of 1,963.75 ms total (92.1%), reflecting the ARM Cortex-A53 CPU’s limited throughput on the Raspberry Pi 3 without GPU acceleration. In the Cloud scenario, the dominant cost is network delay at 574.82 ms (75.0% of total), caused by routing frames over the public internet via Tailscale WireGuard VPN. Notably, the Cloud server ran inference using `CUDAExecutionProvider` on the NVIDIA RTX 5050 GPU, which accounts for its low server-side inference time of 54.86 ms compared to 97.56 ms on the Edge laptop CPU; however, this advantage is entirely eclipsed by the network overhead. In the Edge scenario, all three components are comparatively small: preprocessing is 137.02 ms, network delay is 33.38 ms over local Wi-Fi, and inference is 97.56 ms on the laptop CPU — yielding the best overall latency.

Figure 5.3 shows the per-image latency across all test images. Onboard latency (blue) is the most stable, fluctuating narrowly around ~ 2000 ms throughout the entire evaluation — this consistency reflects that CPU-bound inference is largely deterministic and unaffected by external factors. Edge latency (orange) is both low and stable, remaining mostly below 400 ms. Cloud latency (green) is the most volatile: while the mean is 766.63 ms, individual frames spike to over 3,900 ms (near image 40), and the first 50 images exhibit high variability due to VPN connection warm-up. This instability is a critical operational concern for real-time SAR systems, where consistent low latency is more important than a favourable mean.

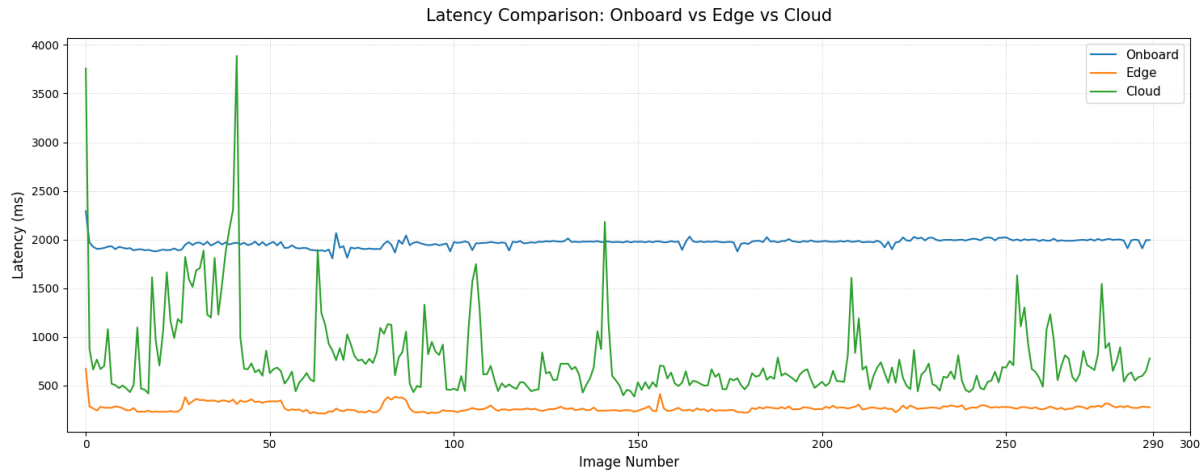


Figure 5.3: Per-image total latency across the VisDrone test images for all three scenarios. Cloud latency exhibits significant spikes due to internet-path variability over the Tailscale VPN.

5.3 Detection Accuracy

Since the same YOLOv8n ONNX model is deployed identically across all three scenarios, detection accuracy is effectively the same across configurations. Table 5.3 presents the aggregate accuracy metrics computed from per-image TP, FP, and FN counts on the VisDrone test split at IoU threshold = 0.50.

Table 5.3: Detection Accuracy Results (VisDrone Test Split, IoU = 0.50)

Metric	All Scenarios
Precision (%)	75.08
Recall (%)	16.05
F1-Score (%)	26.45
True Positives (TP)	2,190
False Positives (FP)	727
False Negatives (FN)	11,454
Total GT annotations	13,644
Images Evaluated	290

The overall precision of 75.08% indicates that approximately three in four of the model’s positive detections are correct — a reliable result given the challenging aerial viewpoint. However, the recall of 16.05% means the model detects only about one in six ground-truth humans across the test set. This is not a model failure but a characteristic of the VisDrone benchmark: the test set images contain 13,644 ground-truth human annotations across 290 images, with many annotations corresponding to very small, densely packed, or heavily occluded pedestrians that

are inherently difficult to detect at a confidence threshold of 0.45. Per-image analysis shows that some images received zero predictions, and the per-image F1 score has a wide distribution, reflecting high variability in scene density and object scale across the VisDrone test set. These constraints motivate future work on small-object detection strategies such as image tiling, higher input resolution, or lower confidence thresholds with post-processing filtering.

5.4 Power Consumption

Power consumption of the Raspberry Pi is a critical metric for drone-based SAR systems, where battery capacity directly constrains mission duration. Power was measured as total energy consumed by the Raspberry Pi 3 Model B (in milliampere-hours, mAh) over the complete evaluation run per scenario, using a USB power meter. Table 5.4 summarises the measurements.

Table 5.4: Raspberry Pi 3 Model B Power Consumption Across Scenarios

Scenario	Total Energy (mAh)	Run Time	Avg Power (W)	Relative to Edge
Onboard	145	12 min 00 s	3.63	2.69×
Edge	54	3 min 45 s	4.32	1.00× (baseline)
Cloud	99	11 min 00 s	2.70	1.83×

The Edge scenario consumed the least total energy (54 mAh) — 2.69× less than Onboard (145 mAh) and 1.83× less than Cloud (99 mAh). Average power draw, calculated as $P = \frac{E_{\text{mAh}} \times 5 \text{ V}}{1000 \times t_{\text{hours}}}$ (USB supply voltage = 5 V), was 3.63 W for Onboard, 4.32 W for Edge, and 2.70 W for Cloud. The Edge scenario shows the highest average draw (4.32 W) but the lowest total energy because its run completed in only 3 min 45 s — the Pi CPU was largely idle during the short offloaded run. The Onboard scenario’s sustained full-CPU utilisation over 12 minutes drives its high total consumption (145 mAh, 3.63 W average). The Cloud scenario’s intermediate figures (99 mAh, 2.70 W) reflect the additional energy cost of maintaining a Tailscale VPN connection over 11 minutes, which draws more power than sustaining a local Wi-Fi connection but less than running sustained on-device inference. As an illustrative example, for a SAR drone equipped with a 5,000 mAh LiPo battery, the Edge scenario would extend mission duration by approximately 2.7× compared to pure Onboard inference — a significant operational advantage.

5.5 Cross-Scenario Comparative Analysis

Table 5.5 summarises all three evaluation dimensions across the three inference scenarios.

Table 5.5: Summary Comparison — Onboard vs. Edge vs. Cloud

Metric	Onboard	Edge	Cloud
Total Latency (ms)	1,963.75	267.95	766.63
Latency Stability	High (deterministic)	High	Low (high variance)
Precision (%)	75.08	75.08	75.08
Recall (%)	16.05	16.05	16.05
F1-Score (%)	26.45	26.45	26.45
Pi Energy (mAh)	145	54	99
Network Required	No	Yes (Wi-Fi)	Yes (Internet / VPN)
Inference Hardware	Pi CPU	Laptop CPU	Laptop GPU

Across all three dimensions, the Edge scenario is the most balanced and practically suitable configuration. It achieves the lowest latency, equivalent detection accuracy across all scenarios, and the lowest power consumption on the Raspberry Pi. The Onboard scenario is operationally necessary when no network infrastructure is available — forest searches, mountain rescues, or post-disaster zones with destroyed communications — and its latency of ~ 2 seconds per frame, while high, remains tolerable for slow-moving drone surveys. The Cloud scenario offers GPU-accelerated inference but its dependence on a stable internet VPN connection and the resulting high and unpredictable latency make it unsuitable for time-critical real-time detection; it is better suited for post-mission batch analysis where latency is not a constraint.

5.6 Suitability for Real-World SAR Operations

Based on the experimental results, the following deployment recommendations are made for real-world SAR drone operations:

- **Time-critical urban SAR (flood, building collapse, crowd search):** The Edge scenario is recommended. Its mean latency of 267.95 ms is sufficient for near-real-time aerial monitoring, and the low Pi power consumption maximises battery-driven mission duration. A lightweight Wi-Fi access point or mobile hotspot at the incident command post is the only infrastructure requirement.
- **Remote and signal-degraded environments (forest, mountain, disaster zone):** The Onboard scenario is recommended. Its complete independence from network infrastructure makes it the only viable option in areas without Wi-Fi or cellular coverage. The ~ 2 -second latency per frame is acceptable for low-speed drone surveys where victims are stationary or slow-moving.
- **Post-mission analysis and high-resolution review:** The Cloud scenario is recommended

when real-time detection is not required. Recorded footage can be batch-processed with GPU-accelerated inference, and the high-latency penalty is irrelevant. This mode also enables use of larger, more accurate YOLO variants (e.g. YOLOv8m or YOLOv8l) that exceed the Raspberry Pi's onboard computational capacity.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This project has presented a comprehensive experimental comparison of three inference architectures — onboard, edge, and cloud — for real-time human detection in Search and Rescue drone systems using a Raspberry Pi 3 Model B as the drone-side platform. Key findings:

- The Edge scenario provides the best balance of latency (267.95 ms), accuracy (Precision 75.08%, F1 26.45%), and power consumption (54 mAh), making it the most suitable architecture for time-critical SAR with local network access.
- The Cloud scenario benefits from GPU-accelerated inference (54.86 ms inference time) but incurs high and variable VPN network overhead (mean 574.82 ms), making it better suited for post-mission analysis.
- The Onboard scenario is the only architecture that operates without network dependency, essential for remote and signal-degraded environments, at a cost of 1,963.75 ms total latency and 145 mAh power consumption.
- ONNX export enables the same trained model to run consistently across CPU (onboard, edge) and GPU (cloud) backends via ONNX Runtime, providing a unified deployment framework.
- The low recall ($\sim 16\%$) observed across all scenarios reflects the inherent difficulty of the VisDrone benchmark rather than a model deficiency, and motivates future work on small-object detection strategies.

6.2 Limitations

- Experiments were conducted under controlled laboratory conditions; real-world network variability may further degrade edge and cloud performance.
- The laptop server is a proxy for cloud infrastructure and may not fully represent production cloud latency.
- The VisDrone dataset contains pre-recorded imagery; real-time drone streaming introduces additional latency not captured here.
- Power was measured as total mAh consumption; per-phase current profiling would provide

finer-grained energy analysis.

6.3 Future Work

- **Onboard NPU acceleration:** Deploy on a Raspberry Pi with a Hailo-8 NPU co-processor to reduce onboard latency substantially.
- **INT8 quantization:** Apply post-training quantization to YOLOv8n ONNX to improve CPU inference throughput on the Pi.
- **Model optimisation:** Explore pruning, layer fusion, and TensorRT-based optimisation techniques to further reduce inference latency and memory usage across onboard, edge, and cloud deployments.
- **Improved recall and accuracy:** Investigate larger YOLO variants, advanced data augmentation, hyperparameter tuning, and additional training data to improve human detection recall and overall detection accuracy.
- **Real-time video streaming:** Extend the current image-based pipeline to support continuous real-time video streaming and frame-by-frame inference for live drone surveillance and search-and-rescue applications.
- **5G cloud integration:** Replace Tailscale VPN with a 5G cellular uplink for realistic remote deployment conditions.
- **Adaptive scenario switching:** Implement a dynamic inference router that switches between onboard, edge, and cloud based on real-time network availability and battery level.
- **Small-object detection:** Investigate image tiling or higher input resolution to improve recall on densely annotated VisDrone scenes.

Bibliography

- [1] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [2] P. Zhu, L. Wen, D. Du, X. Bian, H. Fan, Q. Hu, and H. Ling, “Detection and Tracking Meet Drones Challenge,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 11, pp. 7380–7399, 2022.
- [3] Microsoft, “ONNX Runtime,” 2023. [Online]. Available: <https://onnxruntime.ai/>
- [4] G. Bradski, “The OpenCV Library,” *Dr. Dobbs’s Journal of Software Tools*, 2000.
- [5] Tailscale Inc., “Tailscale: Zero config VPN,” 2023. [Online]. Available: <https://tailscale.com/>
- [6] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proc. IEEE CVPR*, 2016, pp. 779–788.
- [7] S. Han, H. Mao, and W. J. Dally, “Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding,” in *Proc. ICLR*, 2016.
- [8] A. G. Howard *et al.*, “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [9] B. Wu *et al.*, “FBNet: Hardware-Aware Efficient ConvNet Design via Differentiable Neural Architecture Search,” in *Proc. IEEE CVPR*, 2019.
- [10] S. Sambolek and M. Ivasic-Kos, “Automatic Person Detection in Search and Rescue Operations Using Deep CNN Detectors,” *IEEE Access*, vol. 9, pp. 37905–37922, 2021.
- [11] A. Koubaa, A. Ammar, M. Abdelkader, Y. Alhabashi, and L. Ghouti, “AERO: AI-Enabled Remote Sensing Observation with Onboard Edge Computing in UAVs,” *Remote Sensing*, vol. 15, no. 7, p. 1873, 2023.
- [12] A. Koubaa and B. Qureshi, “DroneTrack: Cloud-Based Real-Time Object Tracking Using Unmanned Aerial Vehicles over the Internet,” *IEEE Access*, vol. 6, pp. 13810–13824, 2018.
- [13] P. McEnroe, S. Wang, and M. Liyanage, “A Survey on the Convergence of Edge Computing and AI for UAVs: Opportunities and Challenges,” *IEEE Internet of Things Journal*, vol. 9, no. 17, pp. 15435–15459, 2022.
- [14] D. V. Raviprasad, R. R. Reddy, P. R. S. Reddy, and T. D. S. Kiran, “Autonomous Hu-

- man Detection using Drones in Search and Rescue Operations,” Preprint, SSRN, 2024.
Available: <https://ssrn.com/abstract=5232204>
- [15] M. E. Hossain, M. T. R. Tarafder, N. Ahmed, A. Al Noman, M. I. Sarkar, and Z. Hossain, “Integrating AI with Edge Computing and Cloud Services for Real-Time Data Processing and Decision Making,” *International Journal of Multidisciplinary Sciences and Arts*, vol. 2, no. 4, pp. 252–261, 2023.
- [16] A. Basireddy, K. G. Rao, V. Vaitla, and S. R. Pedda, “Detection of Humans in Search and Rescue Operations Using Ensemble Learning and YOLOv9,” in *Proc. 3rd International Conference on Futuristic Technology (INCOFT 2025)*, vol. 2, pp. 923–928, 2025.